Chapitre 3
Prétraitement des documents textes



Plan

I. Introduction
II. Techniques de prétraitement
III. Vectorisation
IV. Mesures de similarité

I. Introduction

- Traitement du Langage Naturel (Natural Language Processing) est composé de deux parties:
  - Langage Naturel: représente une forme de communication écrite et parlée qui s'est développée de manière naturelle.
  - Traitement: signifie analyser et donner un sens aux données d'entrée avec des ordinateurs.
- NLP vise à apprendre aux machines comment traiter et comprendre le langage des humains.

I. Introduction

- La première phase du système NLP est appelée **prétraitement du texte** (Text preprocessing), qui englobe plusieurs techniques permettant la préparation du corpus pour les prochaines étapes du système. Cette phase indique essentiellement à la machine ce qu'elle doit prendre en compte et ce qu'elle peut ignorer.
- **NLTK** (Natural Language Toolkit) est une bibliothèque de Python dédiée au traitement du langage naturel.

II. Techniques de prétraitement

1. **Lowercasing/uppercasing**
   (Minuscule/Majuscule)
2. **Noise removal** (Suppression du bruit)
3. **Text normalization** (Normalisation du texte)
4. **Stemming** (Dérivé)
5. **Lemmatization** (Lemmatisation)
6. **Tokenization** (Tokenisation)
7. **Stop words** (Mots vides)

II. Techniques de prétraitement

1. Lowercasing/Uppercasing :

La conversion de tous les caractères majuscules existants en minuscules pour que tout le corpus soit en minuscules, ou la conversion de tous les caractères minuscules présents dans le corpus en majuscules pour que tout le corpus soit en majuscules.

II. Techniques de prétraitement

- Entrée:

  « PYTHON est un Langage de Programmation»



- Sortie:

  « python est un langage de programmation»

II. Techniques de prétraitement

2. Noise removal :

- Le bruit est un terme très général et peut signifier les différentes choses perturbant le corpus d'un domaine spécifique.
- Le bruit n'inclut pas seulement les mots, mais peut également inclure des symboles, des signes de ponctuation, des balises HTML, des nombres, des espaces, des mots vides, des termes particuliers, ... etc.

II. Techniques de prétraitement

- La suppression du bruit est cruciale afin que seules les parties importantes des corpus soient introduites dans les modèles, garantissant ainsi des résultats précis.

Avec bruit	Sans bruit
.. python	python
python!!	
#python	
« python »	
<a>python	

II. Techniques de prétraitement

3. Text normalization :

- Il s'agit du processus de conversion d'un corpus brut en une forme canonique et standard, qui vise essentiellement à garantir que l'entrée textuelle est cohérente avant d'être analysée, traitée et exploitée.

II. Techniques de prétraitement

Forme brute	Forme canonique
Spaghetti	Spaghetti
Spagetti	
Spageti	
Spaghetty	
Spagetty	
Dr	Docteur
Prof	Professeur

II. Techniques de prétraitement

- Il n'existe pas de méthode standard de normalisation car elle dépend fortement du corpus et de la tâche à accomplir. La façon la plus courante de procéder consiste à utiliser le mappage de dictionnaire, qui consiste à créer manuellement un dictionnaire qui mappe toutes les différentes formes d'un mot à ce mot, puis à remplacer chacun de ces mots par une forme standard du mot.

II. Techniques de prétraitement

4. Stemming:

- Le stemming est effectué sur un corpus pour réduire les mots à leur forme radicale ou racine.

Avant Stemming	Après Stemming
programming	program
programmer	
programs	

II. Techniques de prétraitement

5. Lemmatization:

- La lemmatisation est un processus qui s'apparente à la recherche de racines : son but est de réduire un mot à sa forme racine. Ce qui le rend différent, c'est qu'il ne se contente pas de couper les extrémités des mots pour obtenir cette forme racine, mais suit plutôt un processus, respecte les règles et utilise souvent **WordNet** pour les mappages afin de renvoyer les mots à leur forme racine.

II. Techniques de prétraitement

- **WordNet** est une base de données de langue anglaise composée de mots et de leurs définitions ainsi que de synonymes et d'antonymes.
- Par exemple, la lemmatisation est capable de transformer le mot « **better** » en son mot la racine « **good** », puisque « **better** » n'est que la forme comparative de « **good** ».

II. Techniques de prétraitement

6. Tokenization:

- La tokenisation est le processus de décomposition d'un corpus en jetons individuels. Les jetons sont les mots les plus couramment utilisés – ce processus décompose ainsi un corpus en mots individuels – mais peuvent également inclure des signes de ponctuation et des espaces, entre autres.

II. Techniques de prétraitement

- Exemple 1:

« Hi! my name is john.»



« Hi» «!» «my» «name» «is» «john» «.»

II. Techniques de prétraitement

- Exemple 2:

« Hi! my name is john.»



« Hi!» «my name is john.»

II. Techniques de prétraitement

7. Stop words:

- Les mots vides sont essentiellement un ensemble de mots couramment utilisés dans n'importe quelle langue.
- Exemples:
  - Déterminants : le, la, les, ...
  - Conjonctions de coordination : pour, un, ni, mais, ou, pourtant, donc, ...
  - Prépositions : dans, sous, vers, avant, ...

II. Techniques de prétraitement

- La raison pour laquelle les mots vides ne sont pas essentiels dans de nombreuses applications est que, si nous supprimons les mots qui sont très couramment utilisés dans une langue donnée, nous pouvons plutôt nous concentrer sur les mots importants.

II. Techniques de prétraitement

- Par exemple, dans le contexte d'un moteur de recherche, si votre requête de recherche est «**how to develop information retrieval applications?**», si le moteur de recherche essaie de trouver des pages Web contenant les termes «**how**», «**to**», «**develop**», «**information**», «**retrieval**», «**applications**», le moteur de recherche trouvera beaucoup plus de pages contenant les termes «**how**», «**to**» que de pages contenant des informations sur le développement d'applications de recherche d'informations, car les termes «**how**» et «**to**» sont si couramment utilisés dans la langue anglaise.

III. Vectorisation

1. Définition
2. Bag of words (Sac de mots)
3. TF-IDF

III. Vectorisation

1°/ Définition:

- Comme mentionné dans le premier chapitre, la phase du prétraitement prépare les données textuelles pour les modèles de ML et DL qui nécessitent des données numériques en entrée, et donc le rôle clé de vectorisation est de transformer les données textuelles prétraitées en données numériques.

III. Vectorisation

2°/ Bag of words :

La manière la plus intuitive et la plus simple de vectoriser les informations textuelles est la suivante :

- attribuer à chaque mot un index entier unique pour construire un dictionnaire de mots ;
- compter le nombre d'occurrences de chaque mot et faire correspondre cette valeur à l'index correspondant.

III. Vectorisation

Exemple:

(doc.1): "John likes to watch movies and Mary likes movies too“

(doc.2): "John also likes to watch football games“



Vocabulaire

['to', 'Mary', 'John', 'movies', 'likes', 'watch', 'and', 'too', 'games', 'also', 'football']



BOW

BOW_1 : [1, 1, 1, 2, 2, 1, 1, 1, 0, 0, 0]

BOW_2 : [1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1]

III. Vectorisation

3°/ TF-IDF:

- **TF-IDF** signifie **Term Frequency Inverse Document Frequency** of records. Il peut être défini comme le calcul de la pertinence d'un mot dans une série ou un corpus par rapport à un texte. La signification augmente proportionnellement au nombre de fois qu'un mot apparaît dans le texte mais est compensée par la fréquence des mots dans le corpus (ensemble de données).

III. Vectorisation

Il se calcule à l'aide la formule suivante:

$$TF - IDF(t,d) = TF(t,d) * IDF(t)$$

tels que:

- T$F(t,\mathrm{d})$: Fréquence du mot t dans le document *d* / Nombre de mots dans *d*

- $IDF(t)$ = log(N/DF(t))

- $DF(t)$: Nombre de documents contenant *t*

- N : Nombre total de documents

III. Vectorisation

Exemple:

(doc.0): "John likes to watch movies and Mary likes movies too“

(doc.1): "John also likes to watch football games“

(doc.2): “Mary don’t likes to watch football games“

Corpus

(doc.0) : "John likes to watch movies and Mary likes movies too“



TF-IDF

TFIDF_0 : [?,?,?,?,?,?,?,?]

III. Vectorisation

(doc.0) : "John likes to watch movies and Mary likes movies too“

Words	TF	IDF	TF-IDF
John	1/10	3/2	0.017
likes	2/10	3/3	0
to	1/10	3/3	0
watch	1/10	3/3	0
movies	2/10	3/1	0.095
and	1/10	3/1	0.047
Mary	1/10	3/2	0.017
too	1/10	3/1	0.047

TFIDF_0 : [0.17,0,0,0,0.095,0.047,0.017,0.047]

IV. Mesures de similarité

1. Définition
2. Exemples des mesures

IV. Mesures de similarité

1°/ <u>Définition:</u>

Mesurer la similarité entre deux phrases consiste à évaluer jusqu'à quel point le sens de ces phrases est proche en termes de proximité de surface "similarité syntaxique" et de signification "similarité sémantique".

IV. Mesures de similarité

2°/ Similarités syntaxiques :

- Distance Euclidienne:

$$SimEucl(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

- Distance Cosinus:

$$SimCosine(x,y) = \frac{\sum_{i=1}^{n} x_i\, y_i}{\sqrt{\sum_{i=1}^{n} x_i^2}\sqrt{\sum_{i=1}^{n} y_i^2}}$$

- Distance Bray-Curtis:

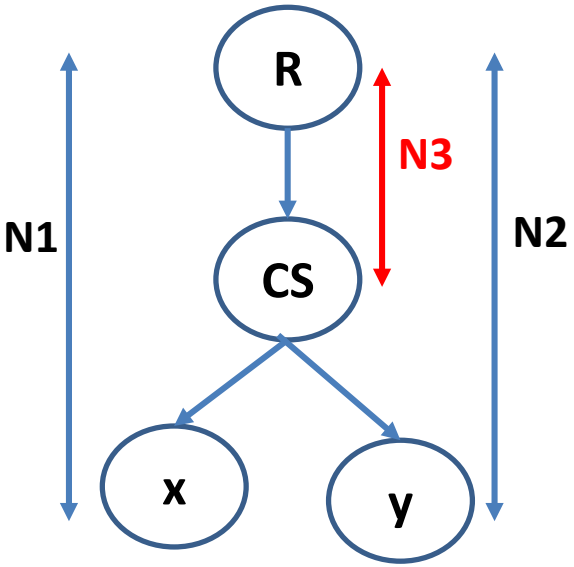
$$SimBray(x,y) = \frac{2\sum_{i=1}^{n}\min(x_i, y_i)}{\sum_{i=1}^{n}(x_i + y_i)}$$

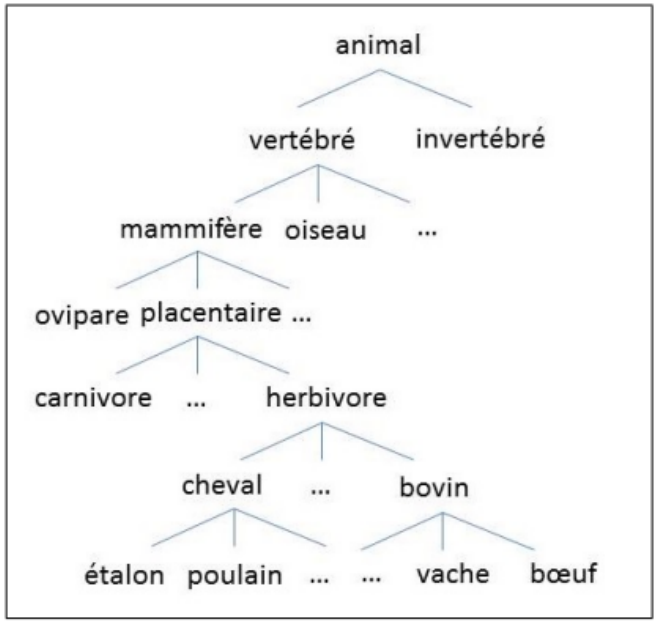
IV. Mesures de similarité

3°/ Similarités sémantiques :

- Similarité de Wu et Palmer:

$$SimWuPalmer(x, y) = \frac{2 \times N_3}{N_1 + N_2}$$





IV. Mesures de similarité

- Similarité de Resnik:

$$SimResnik(c1, c2) = 2 \times D - len(c1, c2)$$

Où :

– D est la longueur maximale des chemins possibles qui connectent *c1* et *c*2, et

– *Len*(*c*1*, c*2) est le plus petit chemin entre c1 et c2.

IV. Mesures de similarité

- Similarité de Leacock et Chodorow:

$$SimLC(c1, c2) = -\log\left(\frac{cd(c1, c2)}{2 \times M}\right)$$

Où :

– M est la longueur du plus long chemin séparant le concept racine de l'ontologie du concept le plus en bas.

– On note cd(c1, c2) la longueur du plus court chemin séparant c1 de c2